WEB-BASED SCHOOL CONCERN AND FEEDBACK REPORTING SYSTEM

SPMP

SOFTWARE PROJECT MANAGEMENT PLAN

Camille Rose Palomaria

Web-based School Concern and Feedback Reporting System

(concern track): and

Version	Primary Author(s)	Description of Version	Date Completed
Draft 1	Wenda D. Panes	First rough draft.	7/1/2017
Draft 2	Wenda D. Panes	Changed Adviser's name, changed project due date, fixed misspelling.	7/8/2017
Version 1	Wenda D. Panes	Changed presentation due date	8/25/2017

Figure **Page**

Figure 1. External Organizational Structure.....47
Figure 2.Internal Structure 48
Figure 3. General Work Breakdown Structure..... 52
Figure 4. Requirements Analysis WBS..... 53
Figure 5. Design Analysis WBS..... 53
Figure 6. Implementation Phase WBS..... 54
Figure 7. Verification Phase WBS..... 54
Figure 8. Maintenance Phase WBS..... 55
Figure 9. Gantt Chart..... 56

Figure 10. Gantt Chart with Resource Allocation 56
Figure 11. Task List with Start and Due Dates..... 57
Figure 12. User Requirements Analysis Task Info..... 58
Figure 13. System Requirements Analysis Task Info..... 58

Figure 14. Prototyping Task Info..... 59

Figure 15. Database Design Task Info..... 59

Figure 16. Front-End Implementation Task Info..... 60

Figure 17. Back-End Implementation Task Info..... 60

Figure 18. White Box Testing Task Info..... 61

Figure 19. User Acceptability Test Task Info..... 61

Figure 20. User Support Task Info..... 62

Figure 21. Documentation Task Info..... 62

Figure 22. Software Development Waterfall Model..... 69

Table **Page**

Table 1. Concern Track Schedule and Budget Allocation Plan 44
Table 2. Concern Track SPMP Evolution Plan 44
Table 3. Definitions 46
Table 4. Concern Track Project Team Roles and Responsibilities 49
Table 5. Concern Track Staff Plan 50
Table 6. Concern Track Non-human resource acquisition plan 54
Table 7. Tasks-Resources-Tools Allocation Plan 82
Table 8. Resource Allocation Plan 83

Table 9. Concern Track Hardware Budget Allocation 64
Table 10. Concern Track Personnel Budget Allocation 65
Table 11. Methods and Techniques Plan 72
Table 12. Software Tools Pla 73
Table 13. Verification and Validation Plan 77

is a web-based digital platform designed to automate the collection ,tracking , and management of the student , teacher , and parent feedback. It replaces traditional paper-based methods with a centralized , accessible online portal that ensures accountability and faster resolution of issues.

1.1. Project Summary

1.1.1. Purpose, scope and objectives

Generally, web-based aims to provide clients and trainers a web interface for information exchange as well as a web-based dashboard for system administration and management tasks. Specifically, objectives are as follows:

- Provide a repository of clients' profile and training needs
- Allow trainers to access training needs information based on criteria set.
- Generate statistical reports on training needs assessment

A web-based training needs assessment and management system that will maintain the Clients, Trainers, Training Areas, Training Programs, Training Enrolment, and Reports; the scope of which is outlined as follows:

A. Web Application
1. Admin privileges:
a. Register qualified trainers
b. Manage the following records:
i. Training Areas (Agriculture, Functional Literacy, Computer Literacy, Livelihood Projects, TESDA Training Programs) ii. Training Programs (Agriculture: Organic Farming, Urban Gardening, Pest Control and Management, Vermicomposting, etc. Computer Literacy: Word Processing, Electronic Spreadsheet, Desktop Publishing, Presentation Packages, Web Development, etc. Livelihood: Trash to Cash, Food Processing, Baking, etc.) iii. Training Schedule
iv. Clients and Trainers
v. Notifications
vi. Reports by:
1. Client Category
2. Location
3. Training Needs Assessment

Client Privileges:

a. Register Profile and Training Needs

b. Select a training program to enroll

c. Receive Notifications 3. Concern Track Privileges:

a. Granted access to client Concern Track needs

b. Provided with printable report based on criteria set.

1.1.2. Assumptions and constraints

Concern Track is planned according to the following assumptions:

- The Software Requirements Specification is well-documented.
- This project is customized for ISCOF-SEC use only.

Concern Track is planned according to the following constraints:

Time: The project shall be completed by the end of September, 2017.

Budget: P50,000 (CHED Assistance)

Staff: The project manager is also the systems analyst, designer, developer and QA tester.

1.1.3. Project deliverables

The deliverables expected for strict compliance are listed below: Project Software and Project Documentation

- A 100% working version of T Concern Track

- Software Requirements Specification (SRS)
- Software Project Management Plan (SPMP)
- Software Design Document (SDD)
- Software Rest Document (STD)
- User Guide/Installation Manual

1.1.4. Schedule and budget summary

Concern track is organized by the following schedule and budget summary reflected in Table 1.

Table1: Concern track Schedule and Budget Allocation Plan

Milestone	Date (Initiation/Target Completion)	Budget
Project Initiation/Proposal	July 10-15, 2025	P 1,000.00
Project Planning	July 16-31, 2025	P 5,000.00
Project Executing	August 1 – 31, 2025	P 40,000.00
Project Monitoring and Controlling	September 1 -30, 2025	P 2, 000.00
Project Closing	October 1-7, 2025	P 2,000.00
Equipment Procurement	November 7-8, 2025	P 54,000.00
	Total	104, 000.00

1.2. Evolution of plan

Table 1:The progress of Concern track SPMP is properly recorded as shown in

Table 2: Concern track SPMP Evolution Plan

Version	Description of Version	Date
Proposal	Overview created for suggestions.	July 15, 2025
First Draft SPMP	First draft integrating the suggestions; distributed for revision.	August 8, 2025
Draft Revision SPMP	Revised as suggested.	August 15, 2025

Final SPMP	First comprehensive SPMP; revision of which is subject to changes made during the actual development.	August 25, 2025
Revision for Final SPMP	Revised according to adviser comments and reviews.	September 2, 2025
Revised Final SPMP	Final packaged SPMP.	September 26, 2025

2. References

[1] R. Stephens, *Beginning Software Engineering*, IN: John Wiley & Sons, Inc., 2015.

[2] F. Tsui and O. Karam, *Essentials of Software Engineering*, MA: Jones and Bartlett Publishers, 2011. Websites

[3]IEEE Std 830-1998 (Revision of IEEE Std 830-1993)

[4]IEEE Std 1058-1998 (Revision and redesignation of IEEE Std 1058.1-1987, incorporating IEEE Std 1058-1998 and IEEE Std 1058a-1998) https://prod.teamgantt.com/gantt

3. Definitions

To properly understand SPMP, terms and acronyms are defined as reflected in

Table 3. Definitions

Term	Definition
CSS	Cascading Style Sheet
CVS	Concurrent Versioning System
DB	Database
GUI	Graphical User Interface
ISCOF-SEC	Iloilo State College of Fisheries – San Enrique Campus
HTML	Hypertext Markup Language
OESCD	Office of Extension Services and Community Development
PHP	Hypertext Preprocessor
QA	Quality Assurance
SPMP	Software Management Plan
SRS	Software Requirements Specification
SDD	Training Management Endeavor
WBS	Work Breakdown Structure
VV	Validation and Verification

4. Project Organization

This section describes interfaces to organizational entities external to web-based project; identifies the internal organizational structure of web- based; and defines the roles and responsibilities of each unit involved in the web-based project.

4.1. External structure

Web-based is developed as requested by the Office of Extension Services and Community Development of ISCOF-SEC to be utilized by the faculty-trainers and training clients in the municipality of San Enrique, Iloilo province. The External Organization Structure Chart is presented in

Figure 1.

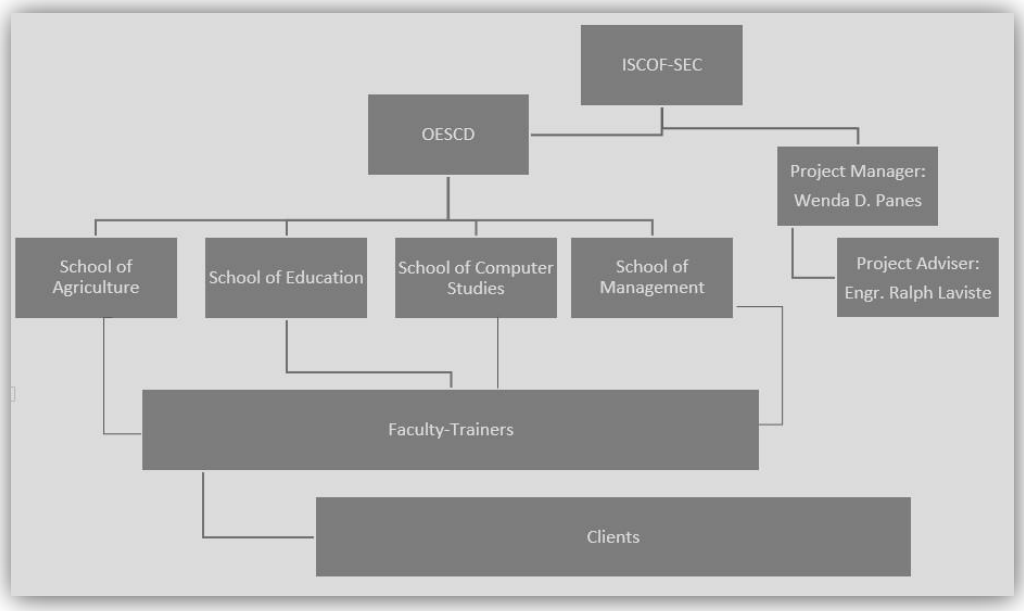


Figure 1 – External Organitional Structure

Functions of the External Structure

The external layer serves several critical functions for the feedback system:

- **Governance and Accountability**: External bodies hold schools responsible for their performance by reviewing feedback data and monitoring whether recommended improvements are implemented.
- **Resource Management**: They may provide funding, specialized training, or technical materials necessary to keep the online system running effectively.
- **Strategic Evaluation**: Independent reviewers often conduct periodic External School Evaluations that incorporate stakeholder views to drive high-level policy changes.

- **Communication Bridge**: The structure facilitates a formal channel for parents and community members to voice concerns without needing to navigate internal school hierarchies directly.

4.2. Internal structure

The web-based project is composed of a one-man army development team that assumes the roles of Project Manager, Program Analyst, Software Developer, Software Tester, Programmer/Designer and User. The lines of authority, responsibility and communication of the aforementioned units are depicted in Figure 2.

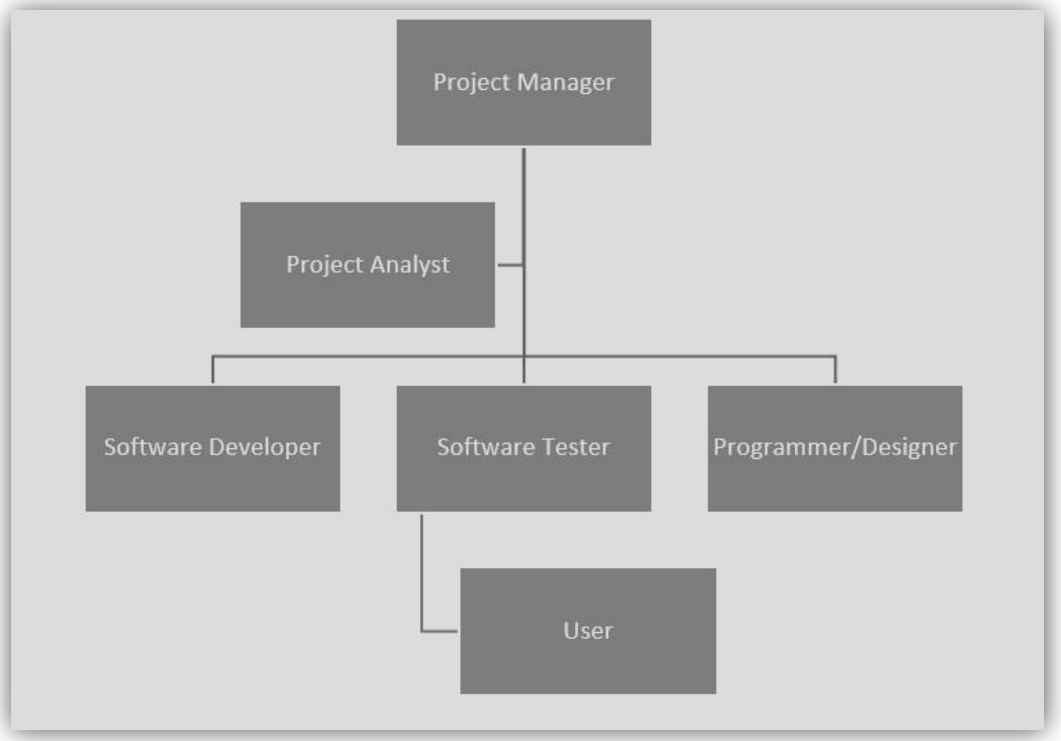


Figure 2. Internal Structure

1. Core Technical Components

Modern implementations often use the **Waterfall methodology** for systematic development and rely on the following tech stack

- **Frontend (User Interface):** Built with (PHP, HTML, CSS, and JavaScript) (often using frameworks like React or Bootstrap) to provide responsive forms for submitting concerns.

- **Backend (Server Side):** Typically powered by PHP or .NET Core to handle business logic, such as routing reports to the correct department.
- **Database (Storage):** A MySQL or PostgreSQL database stores encrypted user credentials and structured feedback records.

2. Database Schema & Key Entities

The internal data structure is relational, linking several key tables to track the lifecycle of

4.3. Roles and responsibilities

The web-based Project is developed by a one-man army person who assumes different responsibilities.

depicts the matrix of such roles and responsibilities.

Table 4. Web-based Project Team Roles and Responsibilities

Role	Responsibilities
Project Manager	Plan, budget, oversee and document all aspects of the specific project. Work closely with upper management. Make sure that the scope and direction of each project is on schedule, as well as other departments for support including status reporting, risk management, escalation of issues that cannot be resolved in the team, and, in general, making sure the project is delivered in budget, on schedule, and within scope.
System Analyst	Analyze, design, and implement information systems. Assess the suitability of information systems terms of their intended outcomes and liaise with end users, software vendors and programmers in order to achieve these outcomes.
QA Tester	Read all the documents and understand what needs to be tested. Based on the information procured in the above step decide how it is to be tested. Inform the test lead about what all resources will be required for software testing. Develop test cases and prioritize testing activities. Execute all the test case and the report defects, define severity and priority for each defect. Carry out regression testing every time when changes are made to the code to fix defects.

Document Controller / Technical Writer	Prepares process and status reports; applies metrics to analyze and asses efficiencies of current Document Control processes. Reviews change control documentation for accuracy and completeness. Coordinates the project documents and their review by all team members. Collects, proofreads, and integrates document parts. Generates the final version of all the documents.
Programmer	Control trial runs of programs and software applications to be sure they will produce the desired information and that the instructions are correct. Write update and maintain computer programs or software packages to handle specific jobs, retrieving data.
User	Use the deliverables of the project. Test the usability of the project.

5. Managerial process plans

This clause of the SPMP specifies the Tra web based management processes for the project. It includes the project start-up plan, risk management plan, project work plan, project control plan, and project closeout plan.

5.1. Start-up plan

The web-based start-up plan specifies the estimation plan, staffing plan, resource acquisition plan, and training plan. These plans may be incorporated directly or by reference to other plans.

5.1.1. Staffing plan

The web-based Staff Plan depicted in Table 5 shows the name, affiliation to the project as well as the status of staff involved in the project.

Table 5. Web-based Staff Plan

Name	**Affiliation to project**	**Status s July 2025**	**Email Address**

Wenda D. Panes	Project Manager	Full Time	wendapanes@gmail.com
Wenda D. Panes	System Analyst	Full Time	wendapanes@gmail.com
Wenda D. Panes	Developer/Programmer	Full Time	wendapanes@gmail.com
Wenda D. Panes	QA	Full Time	wendapanes@gmail.com
Wenda D. Panes	Document Controller	Full Time	wendapanes@gmail.com
Engr. Ralph P. Laviste	Adviser	Full Time	rlaviste@gmail.com

5.1.3. Resource acquisition plan

For the duration of web-based project development, hiring of additional human resource is not necessary. The Project manager who will assume multi-roles will be the sole responsible to deliver both software and documents requirement. Non-human resources however are needed. The description and dates required for each resource are reflected in

Table 6. Non-human resource acquisition plan

Resources	Required Dates	Description
Computer for Implementation	July 2025 – October 2026	At least 2 computers in the laboratory, and faculty room may be made available to be utilized by the developer.
Network equipment & devices	July 2025– October 2026	Switches, wireless routers, and internet facility both in the laboratory and faculty room will be utilized by the developer as scheduled.
Software / Domain Purchase	October 2026	A web domain will be purchased to launch the live in the cloud. This will be taken care of by the Procurement Office.

Printing Services	July 2025 – October 2026	Sample system-generated documents may be printed for verification purposes. Printing facilities of laboratory and faculty room may be utilized upon request.
Data Repository	July 2025 – October 2026	Bulk of accumulated data will need an ample storage space. One of computer unit will be utilized solely for data server purposes.

5.1.4. Project staff training plan

The developer is already experienced in system analysis and design. Likewise, a substantial training on management has been participated. On technical aspects, however, a refresher course will be needed to update knowledge and skills. Online courses and tutorials will be taken every Saturdays and Sundays. The OESCD personnel will be available to cater to the developer inquiries in vague areas. The project adviser will guide the developer all throughout the execution of project phases so as to guarantee completion of project on time and within budget.

5.2. Work plan

This section describes in detail the decomposition of the total scope of work of the project to be carried out by the project team to accomplish the project objectives and create the required deliverables.

5.2.1. Work activities

The Work Breakdown Structure (WBS) is created using the Match Ware Mind View 7.0 Software. It depicts the work activities and the relationships among work activities. Work activities are decomposed to a level that allows accurate estimate of resource requirements and schedule duration for each work activity. The WBS Top-Down Map View is presented in Figure

3. Activity listings and deliverables for each phase are shown in succeeding figures.

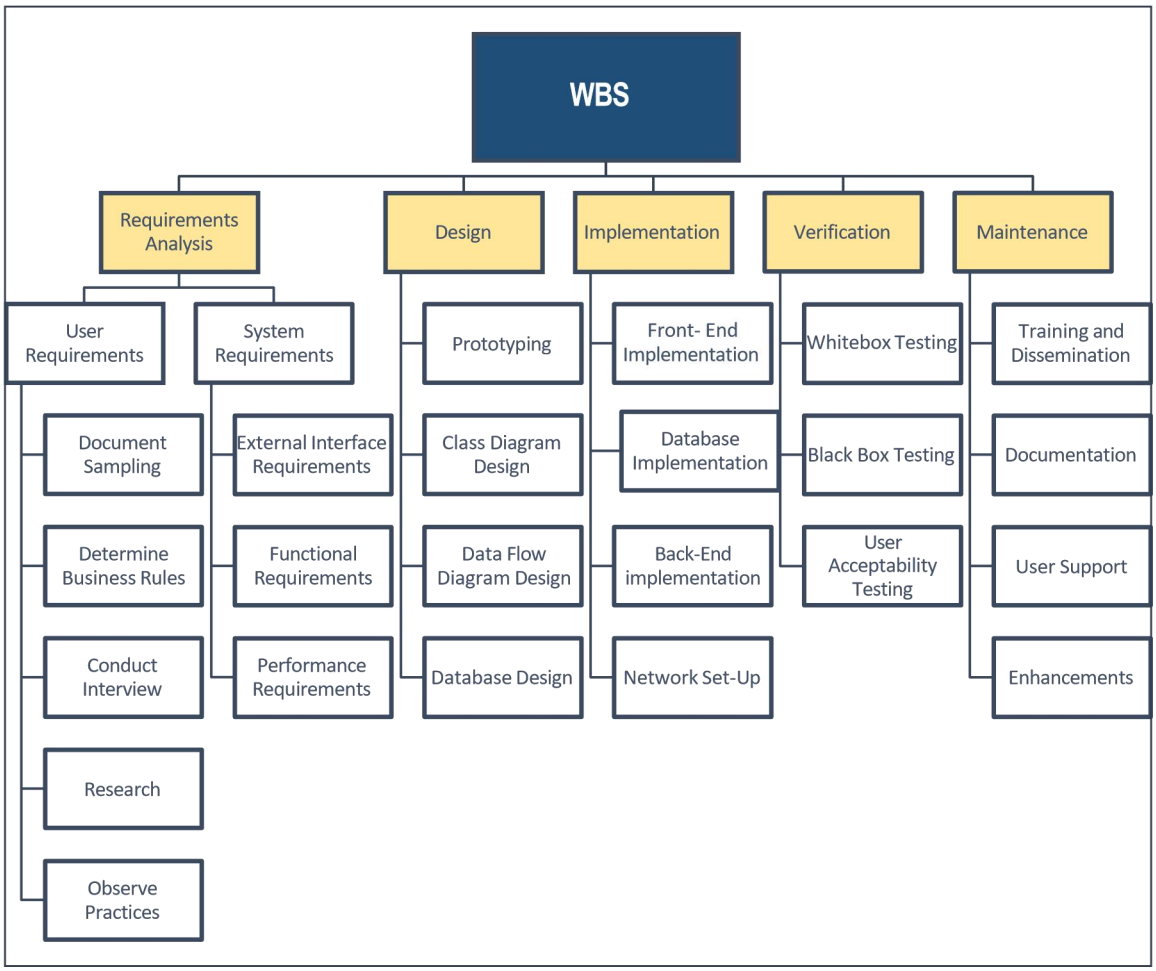


Figure 3. General Work Breakdown Structure

1. Core Technical Components

Modern implementations often use the **Waterfall methodology** for systematic development and rely on the following tech stack

- **Frontend (User Interface):** Built with HTML, CSS, and JavaScript (often using frameworks like React or Bootstrap) to provide responsive forms for submitting concerns.
- **Backend (Server Side):** Typically powered by PHP or .NET Core to handle business logic, such as routing reports to the correct department.

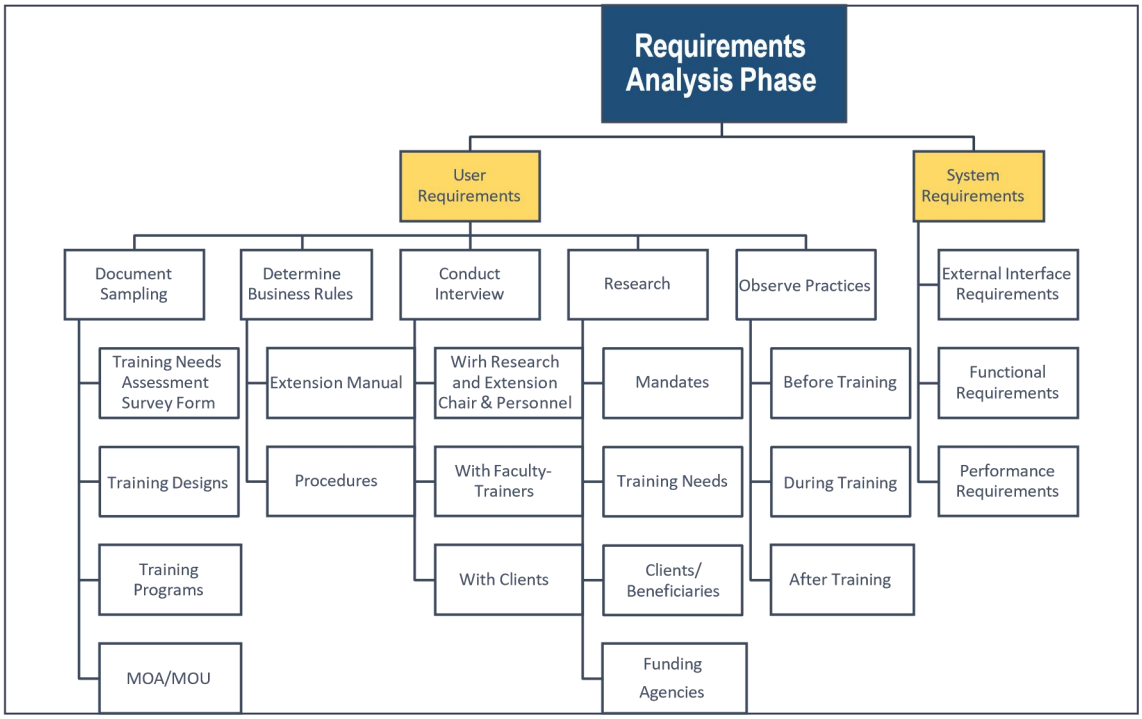


Figure 4. Requirements Analysis WBS

Typical WBS Breakdown: Requirements Analysis

Below is an example of how this phase is decomposed into smaller, manageable work packages:

- **2.1 Stakeholder Elicitation** ○ 2.1.1 Stakeholder Identification & Categorization ○ 2.1.2 User Interviews & Workshops ○ 2.1.3 Business Objective Documentation
- **2.2 Functional & Non-Functional Requirements** ○ 2.2.1 Core Feature List (Functional)

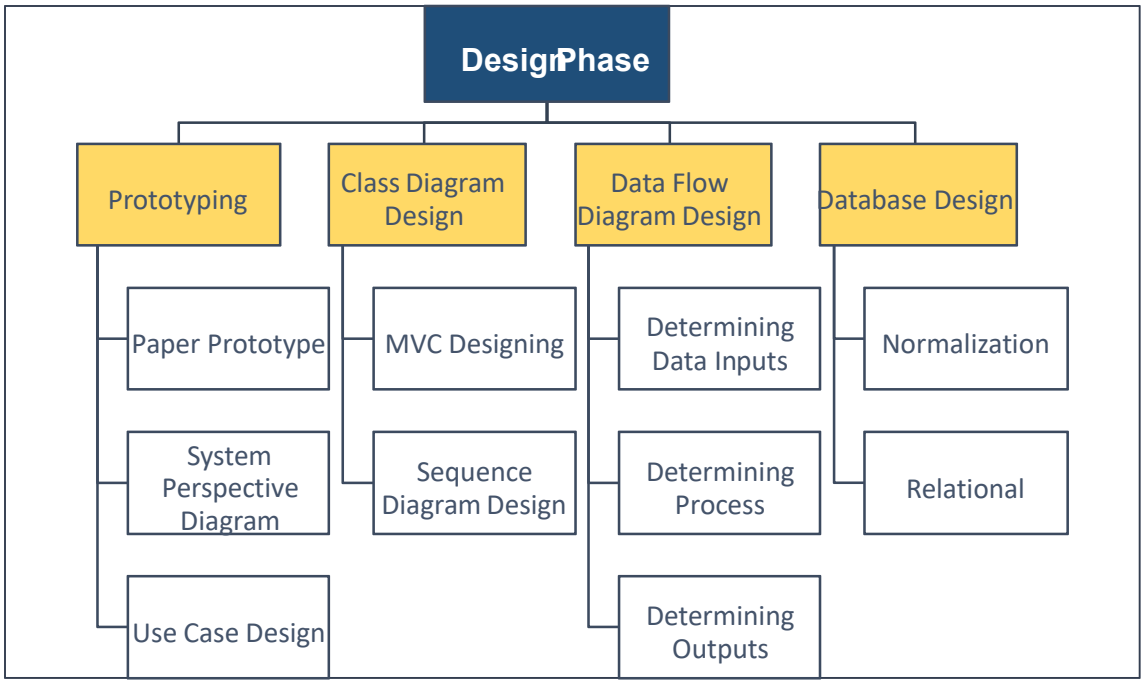


Figure 5. Design Phase WBS

The **Design Phase** of a web-based Work Breakdown Structure (WBS) transforms user requirements into visual and architectural blueprints. This phase is deliverable-oriented, ensuring that the development team has a clear roadmap for the build.

WBS Breakdown: Design Phase (Level 2 & 3)

A standard design phase is typically broken down into these core work packages:

- **3.1 Information Architecture (IA)** ○ 3.1.1 Site Map Generation ○ 3.1.2 User Flow Diagrams ○ 3.1.3 Content Hierarchy & Navigation Logic

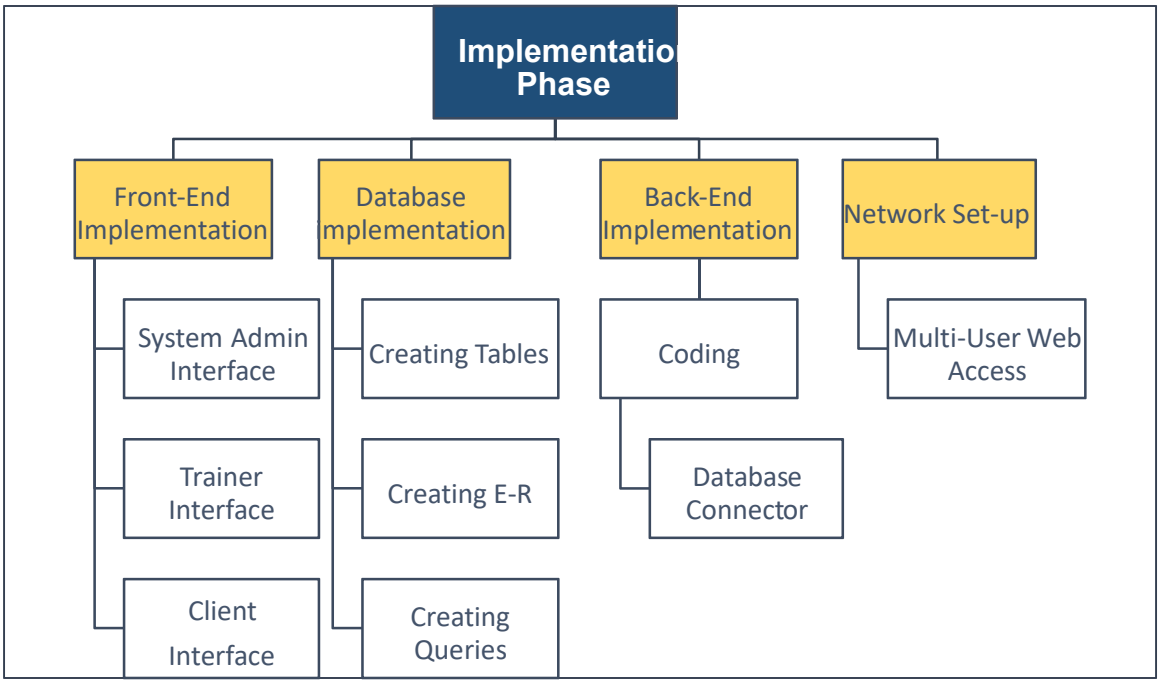


Figure 6. Implementation Phase WBS

The **Implementation Phase** (often called the Development Phase) is the "build" stage of the project where designs and requirements are converted into a functional web application. In a web-based Work Breakdown Structure (WBS), this phase focuses on coding, technical integration, and establishing the foundational environment. [1, 2, 3, 4]

WBS Breakdown: Implementation Phase (Level 2 & 3)

A standard implementation phase for a web project typically includes these work packages: [1]

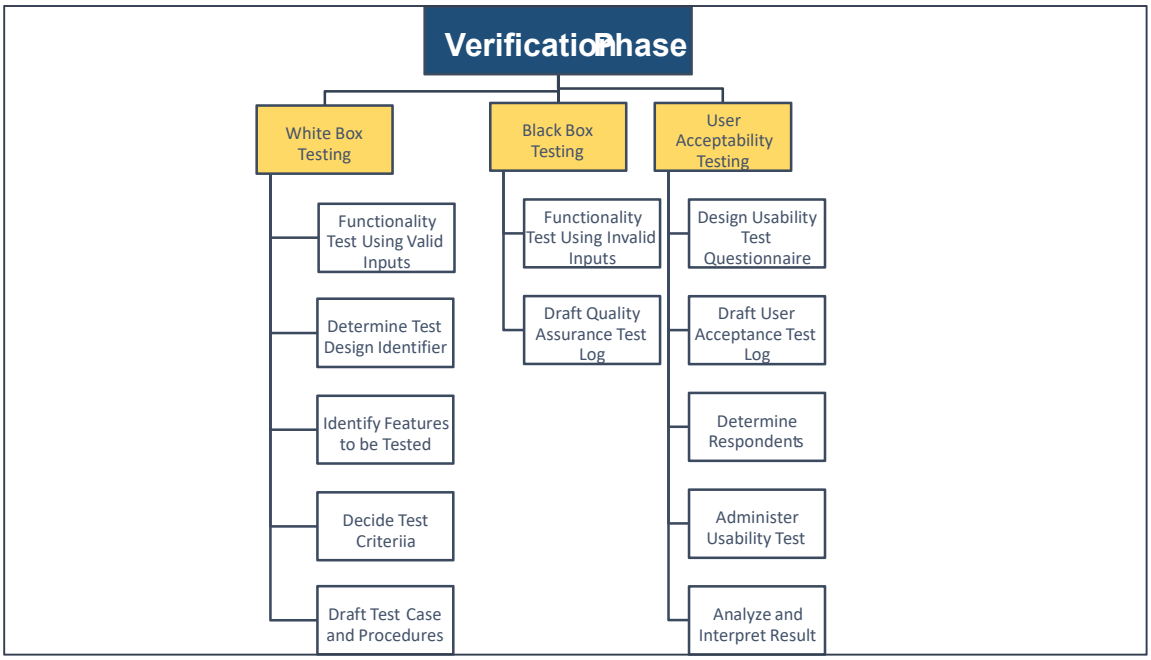
- **4.1 Environment & Infrastructure Setup** ○ 4.1.1 Development, Staging, and Production Environment SetP

4.4.2 Third-Party API & Service Integration (e.g., Payments, CRM) ○ 4.4.3 Data Migration from Legacy Systems (if applicable) [1, 2, 3, 4, 5, 6]

Key Implementation Deliverables

These outputs represent the tangible "work packages" completed during this phase: [1]

- **Functional Codebase:** Clean, maintainable code pushed to the main repository.
- **Database Schema:** A fully implemented and populated database structure.
- **Technical Documentation:** Inline code comments and external system architecture guides.
- **Integrated API:** Fully documented endpoints ready for frontend or third-party



1. **Test Planning** This involves reviewing system requirements, defining the overall test strategy, allocating resources, and preparing the test environment. The goal is to establish a clear roadmap for verification activities.
2. **Test Case Development** Detailed test cases are created for unit, integration, system, and user acceptance testing. Each case ensures that modules such as login, feedback submission, concern categorization, and administrative dashboards are validated against expected outcomes.
3. **Test Execution** Testing is carried out at multiple levels: unit testing verifies individual modules, integration testing checks interactions between subsystems, system testing validates end-to-end workflows, and user acceptance testing confirms usability and satisfaction among students, teachers, and administrators.
4. **Defect Management** Any issues discovered during testing are logged, tracked, and resolved. Retesting is performed to confirm fixes and ensure no regression occurs, maintaining the integrity of the system.
5. **Verification Reporting** A comprehensive test summary report is produced, documenting results, coverage, and defects. Quality metrics are analyzed to measure performance, reliability, and usability. Finally, a formal sign-off is obtained to approve the system's readiness for deployment.

Figure 7. Verification Phase

Figure 8. Maintenance

Phase WBS

1. **Corrective Maintenance** ○ Identify and fix defects or errors discovered after deployment. ○ Activities include bug tracking, troubleshooting, and applying patches to restore normal operations.
2. **Adaptive Maintenance** ○ Adjust the system to remain compatible with evolving environments such as new operating systems, browsers, or school IT infrastructure.
    - Ensures continued functionality as technology changes.
3. **Perfective Maintenance** ○ Introduce enhancements to improve usability, performance, and efficiency. ○ Examples include refining the feedback submission interface, optimizing database queries, or adding new reporting features requested by administrators.
4. **Preventive Maintenance** ○ Take proactive measures to reduce the likelihood of future issues. ○ Includes code refactoring, database optimization, security audits, and regular system health checks.
5. **Maintenance Reporting** ○ Document maintenance activities, including logs of issues resolved, updates applied, and performance metrics.
    - Reports are shared with stakeholders to ensure transparency and accountability.

5.2.2. Schedule allocation

Figure 9 shows the web-based schedule allocation plan designed in Microsoft Excel. The project duration is 65 working days with a starting date on July 3, 2017 and expected to end on October 6, 2017. Moreover, Gantt charts and activity lists created in Gantt chart further depict the detailed schedule allocation with milestones and assigned resource for each project phase. Selected Gantt chart screen grabs of individual task management are also presented in the following figures.

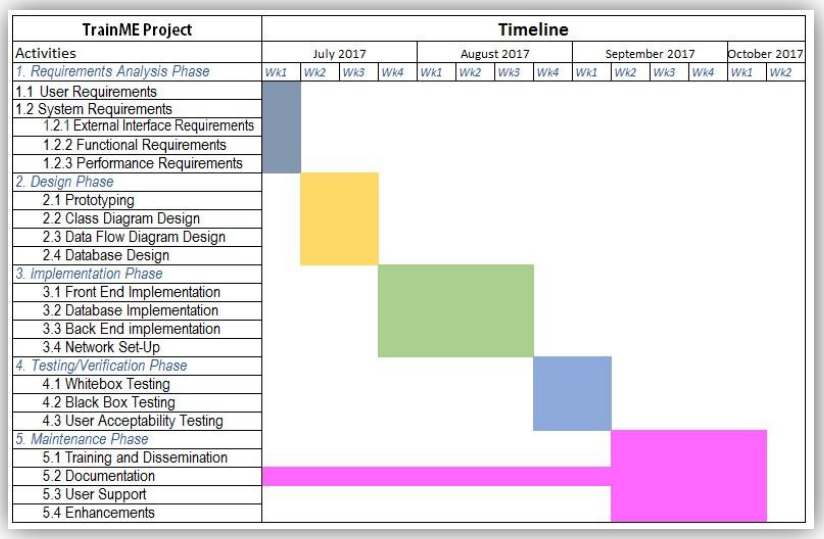


Figure 9. Gantt Chart

Phase / Task	Duration	Start – End	Dependencies	Resources
Requirement Analysis	5 days	May 20 – May 24, 2026	None	Project Manager, Analyst
System Design	7 days	May 25 – May 31, 2026	Requirement Analysis	System Architect, QA Lead
Prototyping	10 days	June 1 – June 10, 2026	System Design	UI/UX Designer, Developers
Database Design	7 days	June 11 – June 17, 2026	System Design	Database Administrator, Developers
Front-End Implementation	14 days	June 18 – July 1, 2026	Prototyping	Front-End Developers
Back-End Implementation	14 days	June 18 – July 1,	Database Design	Back-End Developers

		2026		
White Box Testing	10 days	July 2 – July 11, 2026	Implementation	QA Engineers
User Acceptability Testing	7 days	July 12 – July 18, 2026	White Box Testing	QA Engineers, End Users
Deployment	7 days	July 19 – July 25, 2026	UAT	DevOps Engineer, IT Support
User Support	Ongoing	July 26, 2026 – onwards	Deployment	Help Desk, Trainers

Phase / Task	Duration	Start – End	Dependencies	Resources
Documentation	Continuous	May 20 – July 25, 2026	All Phases	Project Team
Maintenance	Ongoing	Post-deployment	Deployment	Developers, QA, DBA, Security Analyst

Figure 10. Gantt Chart with Resource Allocation

Phase / Task	Duration	Start – End	Dependencies	Resources
Requirement Review	5 days	Week 1	None	Project Manager, Analyst

System Design Verification	7 days	Week 2	Requirement Review	System Architect, QA Lead
Test Case Development	10 days	Week 3–4	Design Verification	QA Engineers
Test Execution	15 days	Week 5–7	Test Case Development	QA Engineers, Developers
Defect Management	Continuous	Week 5–8	Test Execution	QA Engineers, Developers
Verification Reporting	5 days	Week 8	Test Execution	QA Lead, Project Manager
Deployment	7 days	Week 9	Verification Reporting	DevOps Engineer, IT Support
Corrective Maintenance	Ongoing	Post deployment	Deployment	Developers, QA Engineers
Adaptive Maintenance	Ongoing	Postdeployment	Deployment	Developers, System Admin
Perfective Maintenance	Ongoing	Postdeployment	Deployment	Developers, UX Designer
Phase / Task	**Duration**	**Start – End**	**Dependencies**	**Resources**
Preventive Maintenance	Scheduled (Quarterly)	Post-deployment	Deployment	Security Analyst, DBA

📌 Resource Allocation Summary

- **Project Manager** → Oversees planning, reporting, and sign-off.
- **System Architect** → Ensures design aligns with requirements.
- **QA Engineers** → Develop and execute test cases, manage defects.
- **Developers** → Fix bugs, implement patches, adapt system.
- **DevOps Engineer** → Handles deployment and environment setup.
- **IT Support** → Provides technical assistance during rollout.
- **UX Designer** → Improves usability and interface.
- **Security Analyst & DBA** → Conduct preventive maintenance (security audits, database optimization).

9/7/2017 TrainME | TeamGantt

TrainME

Requirements Analysis Phase	**100%**	**Start**	**Due**	**Assigned**
User Requirements Analysis	100%	Jul 4, 2017	Jul 7, 2017	S...
System Requirements Analysis	100%	Jul 6, 2017	Jul 7, 2017	...
Client Approval		Jul 3, 2017	Jul 3, 2017	...
Training Needs Assessment		Jul 3, 2017	Jul 3, 2017	...
Design Phase	**100%**	**Start**	**Due**	**Assigned**
Prototyping	100%	Jul 10, 2017	Jul 11, 2017	...
Class Diagram Design	100%	Jul 12, 2017	Jul 13, 2017	S...
Data Flow Design	100%	Jul 14, 2017	Jul 17, 2017	...
Database Design	100%	Jul 18, 2017	Jul 21, 2017	...
Implementation Phase	**100%**	**Start**	**Due**	**Assigned**
Front-End Implementation	100%	Jul 24, 2017	Aug 4, 2017	...
Database Implementation	100%	Jul 24, 2017	Aug 11, 2017	...
Back-End Implementation	100%	Jul 24, 2017	Aug 25, 2017	...
Network Set-Up	100%	Jul 24, 2017	Aug 25, 2017	...
Verification Phase	**100%**	**Start**	**Due**	**Assigned**
White Box Testing	100%	Aug 28, 2017	Aug 30, 2017	...
Black Box Testing	100%	Aug 31, 2017	Sep 1, 2017	S...
User Acceptability Test	100%	Sep 4, 2017	Today	...
Maintenance Phase	**43%**	**Start**	**Due**	**Assigned**
Training and Dissemination	0%	Tomorrow	Sep 15, 2017	...
Documentation	50%	Jul 3, 2017	Sep 29, 2017	S...
User Support	0%	Oct 2, 2017	Oct 3, 2017	...
Enhancements	0%	Oct 4, 2017	Oct 6, 2017	...

https://prod.teamgantt.com/gantt/list/?ids=899603#&ids=899603&user=&custom=&company=&hide_completed=false&date_filter=&color_filter= 1/2

Figure 11. Task List with Start and Due Dates

Task	Description	Start Date	Due Date	Resources

Requirement Review	Analyze functional and non-functional requirements.	May 20, 2026	May 24, 2026	Project Manager, Analyst
System Design Verification	Validate design against requirements.	May 25, 2026	May 31, 2026	System Architect, QA Lead
Test Case Development	Create unit, integration, system, and acceptance test cases.	June 1, 2026	June 10, 2026	QA Engineers
Test Execution	Perform unit, integration, system, and	June 11,	June 25,	QA Engineers,

Task	Description	Start Date	Due Date	Resources
	acceptance testing.	2026	2026	Developers
Defect Management	Log, track, and resolve defects during testing.	June 11, 2026	June 28, 2026	QA Engineers, Developers
Verification Reporting	Document test results, coverage, and quality metrics.	June 26, 2026	June 30, 2026	QA Lead, Project Manager
Deployment	Roll out the system to production environment.	July 1, 2026	July 7, 2026	DevOps Engineer, IT Support
Corrective Maintenance	Fix bugs and errors reported post-deployment.	July 8, 2026	Ongoing	Developers, QA Engineers
Adaptive Maintenance	Ensure compatibility with new environments.	July 8, 2026	Ongoing	Developers, System Admin
Perfective Maintenance	Enhance usability, performance, and features.	July 8, 2026	Ongoing	Developers, UX Designer
Preventive Maintenance	Conduct audits, optimizations, and security checks.	July 8, 2026	Quarterly	Security Analyst, DBA

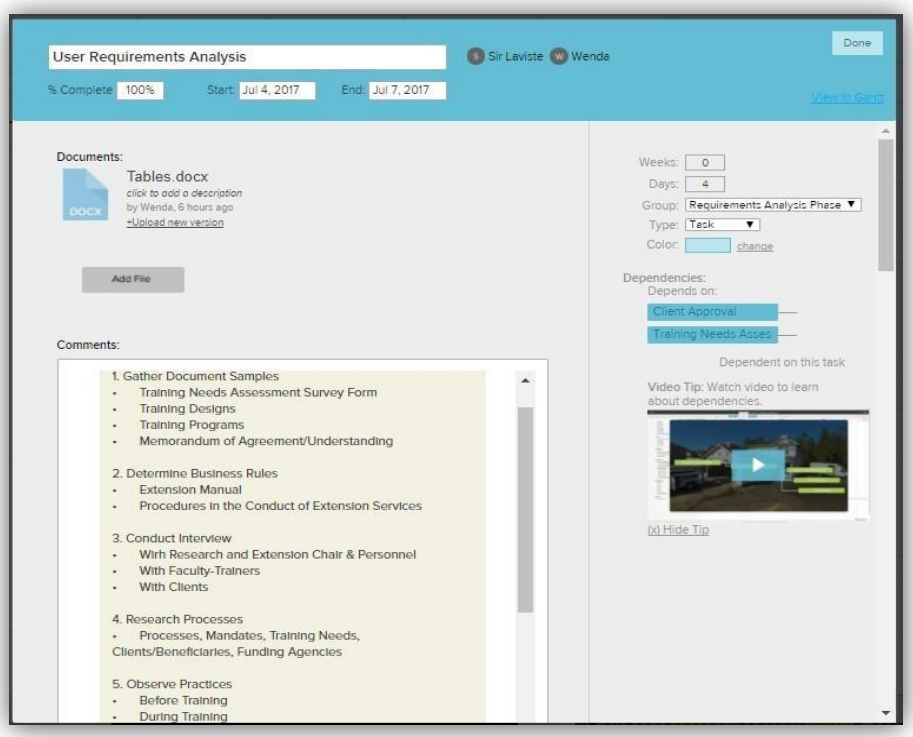


Figure 12. User Requirements Analysis Task Info

1. **Requirement Gathering**
   Collect inputs from stakeholders (students, teachers, administrators, parents) through surveys, interviews, and focus groups to identify expectations and pain points in the current feedback process.
2. **Requirement Documentation**
   Record functional requirements (e.g., feedback submission, concern categorization, reporting dashboards) and non-functional requirements (e.g., security, usability, performance) in a structured format.
3. **Requirement Classification**
   Categorize requirements into essential, desirable, and optional features. This helps prioritize development efforts and resource allocation.
4. **Requirement Validation**
   Review documented requirements with stakeholders to confirm accuracy, feasibility, and alignment with school policies and IT infrastructure.
5. **Requirement Approval**

Obtain formal sign-off from school management and project sponsors to ensure consensus and commitment before moving to design and development.

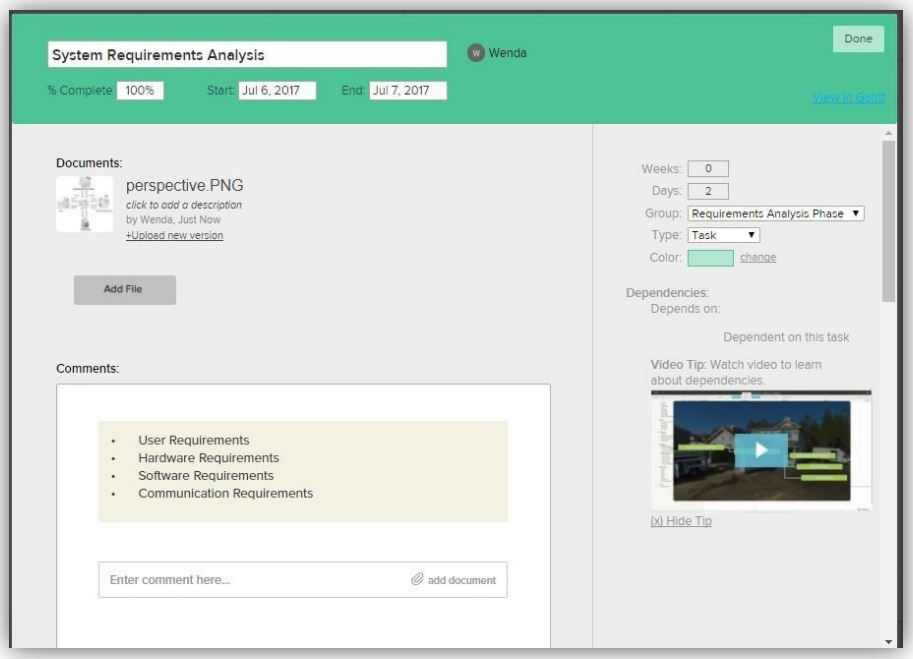


Figure 13. System Requirements Analysis Task Info

1. **Functional Requirements Analysis**
   Define the core functionalities such as user authentication, feedback submission, concern categorization, administrative dashboards, and reporting tools. These requirements specify what the system must do to support students, teachers, and administrators.
2. **Non-Functional Requirements Analysis**
   Identify performance, usability, security, and reliability standards. Examples include response time, data encryption, accessibility compliance, and system availability.
3. **Technical Requirements Specification**
   Document hardware, software, and network requirements. This includes server capacity, database management systems, browser compatibility, and integration with existing school IT infrastructure.
4. **Interface Requirements**

Define how users interact with the system, including web-based portals for students and teachers, mobile responsiveness, and administrator dashboards.

5. **Data Requirements**
   Specify how data will be collected, stored, processed, and reported. This includes feedback records, concern categories, resolution tracking, and compliance with privacy policies.
6. **Validation and Approval**
   Review requirements with stakeholders to ensure accuracy, feasibility, and alignment with school policies. Obtain formal approval before moving to design and development.

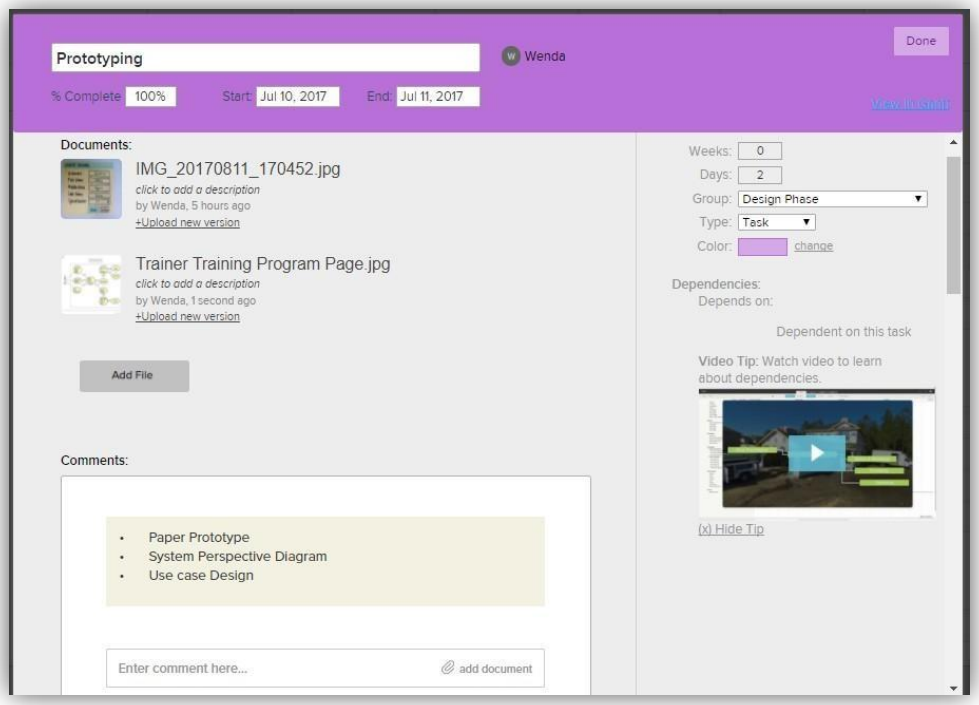


Figure 14. Prototyping Task Info

1. **Prototype Planning**
   Define the scope, objectives, and features to be included in the prototype. Decide whether to build low-fidelity (wireframes) or high-fidelity (interactive mockups) prototypes depending on stakeholder needs.
2. **Interface Design**
   Develop user interface layouts for student portals, teacher dashboards, and administrator panels. Focus on usability, accessibility, and intuitive navigation.

3. **Prototype Development**
   Build functional mockups using web technologies or prototyping tools. Include core features such as login, feedback submission, concern categorization, and reporting modules.
4. **Stakeholder Review**
   Present the prototype to students, teachers, and administrators. Collect feedback on usability, clarity, and alignment with requirements.
5. **Prototype Testing**
   Conduct usability testing sessions to identify design flaws, workflow inefficiencies, or missing features. Ensure the prototype reflects real-world usage scenarios.
6. **Refinement and Iteration**
   Incorporate stakeholder feedback and test results into successive versions of the prototype. Iterate until the design meets both functional and non-functional requirements.
7. **Approval and Transition**
   Obtain formal approval from school management and project sponsors. Transition the validated prototype into the system design and development phase.

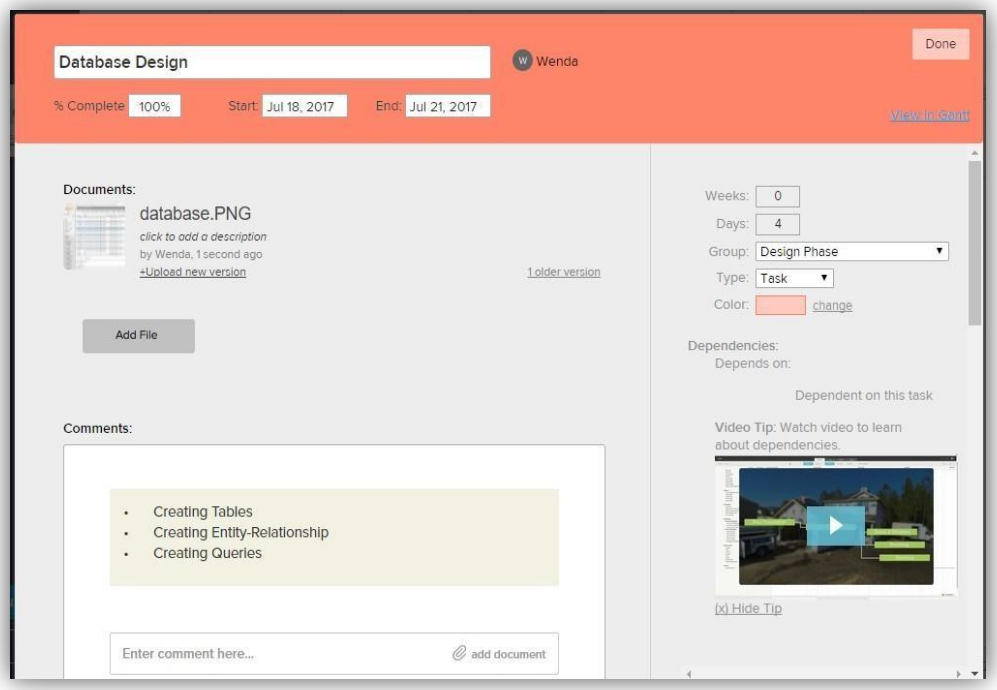


Figure 15. Database Design Task Info

1. **Requirement Analysis**
   Identify the data needs of the system, including student records, feedback entries, concern categories, resolution tracking, and administrative reports.
2. **Conceptual Design**
   Develop an Entity-Relationship Diagram (ERD) to represent entities such as *Students*, *Teachers*, *Administrators*, *Feedback*, *Concerns*, and *Reports*, along with their relationships.
3. **Logical Design**
   Translate the conceptual model into relational schemas. Define tables, attributes, primary keys, foreign keys, and normalization rules to eliminate redundancy and maintain data integrity.
4. **Physical Design**
   Specify how the database will be implemented on the chosen DBMS (e.g., MySQL, PostgreSQL). Define indexing strategies, storage structures, and optimization techniques for performance.
5. **Security Design**
   Implement access control, authentication, encryption, and backup strategies to protect sensitive student and faculty data.
6. **Testing and Validation**
   Test the database with sample data to ensure accuracy, efficiency, and compliance with requirements. Validate queries, reports, and data integrity constraints.
7. **Documentation**
   Prepare detailed documentation of database schemas, relationships, and procedures for developers, administrators, and future maintenance.

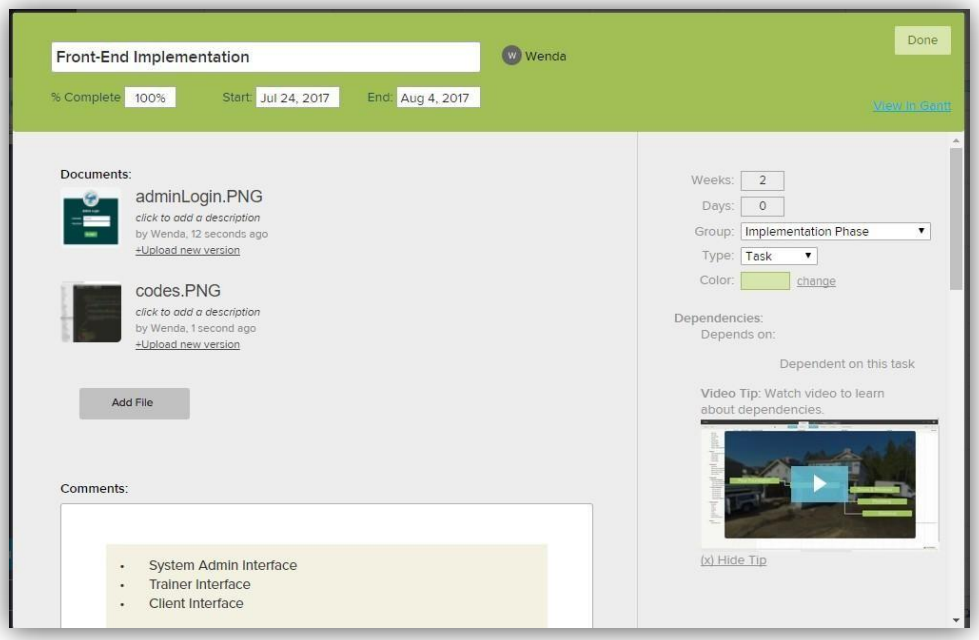


Figure 16. Front-End Implementation Task Info

1. **UI Framework Selection**
   Choose appropriate front-end technologies (e.g., HTML5, CSS3, JavaScript, React, Angular, or Vue.js) to support responsive and scalable design. 2. **Layout Design**
   Create consistent layouts for student portals, teacher dashboards, and administrator panels. Ensure intuitive navigation and accessibility compliance.
3. **Component Development**
   Build reusable components such as login forms, feedback submission modules, concern categorization dropdowns, and reporting charts.
4. **Responsive Design**
   Implement mobile-friendly and cross-browser compatibility features to ensure smooth access on desktops, tablets, and smartphones.
5. **Integration with Backend**
   Connect front-end components with backend services via APIs to enable data submission, retrieval, and reporting.
6. **Testing and Debugging**
   Conduct usability testing, cross-browser testing, and performance checks. Debug issues related to layout, responsiveness, or data flow.
7. **User Feedback Incorporation**

Gather input from students, teachers, and administrators during pilot runs. Refine the interface based on usability and accessibility feedback.

8. **Documentation**
   Prepare detailed documentation of front-end components, workflows, and integration points for developers and future maintenance.

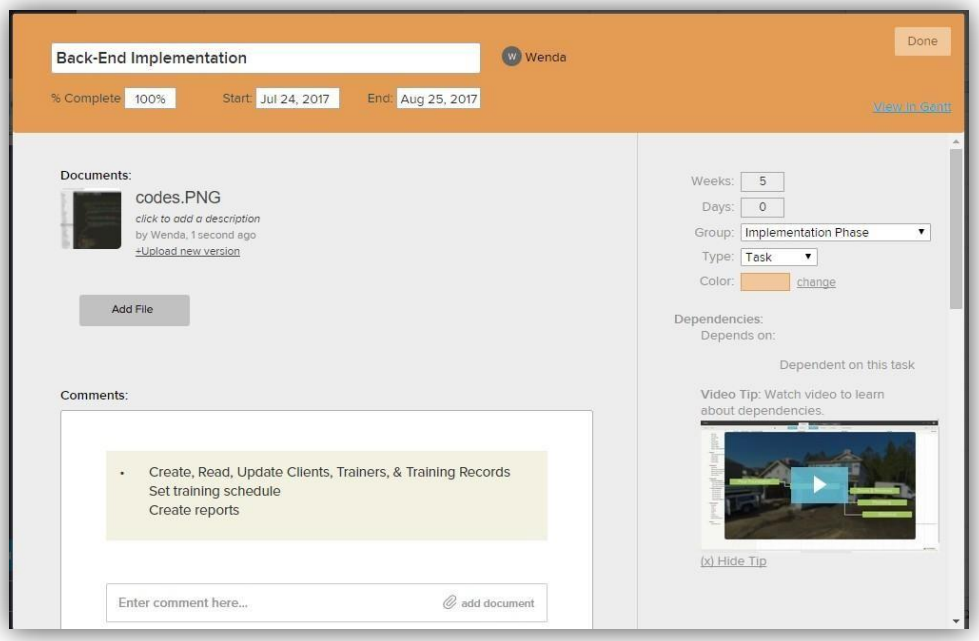


Figure 17. Back-End Implementation Task Info

1. **Framework and Language Selection**
   Choose appropriate back-end technologies (e.g., PHP with MySQLi, Node.js with Express, or Python with Django) based on scalability, security, and compatibility with school IT infrastructure.
2. **API Development**
   Create RESTful or GraphQL APIs to handle communication between the front-end and back-end. APIs manage functions such as feedback submission, concern categorization, and report generation.

3. **Database Integration**
   Implement secure connections to the database. Ensure CRUD (Create, Read, Update, Delete) operations are optimized for student records, feedback entries, and administrative reports.
4. **Authentication and Authorization**
   Develop secure login and role-based access control for students, teachers, and administrators. Implement encryption and session management to protect sensitive data.
5. **Business Logic Implementation**
   Encode workflows such as categorizing concerns, routing feedback to appropriate staff, and generating resolution timelines. Ensure automation reduces manual workload.
6. **Error Handling and Logging**
   Implement robust error detection, exception handling, and logging mechanisms to track system issues and support corrective maintenance.
7. **Security Measures**
   Apply input validation, SQL injection prevention, encryption protocols, and regular vulnerability checks to safeguard system integrity.
8. **Testing and Debugging**
   Conduct unit testing, integration testing, and performance testing to ensure reliability. Debug issues related to API responses, database queries, and authentication.
9. **Documentation**
   Prepare technical documentation covering APIs, workflows, security protocols, and integration points for developers and administrators.

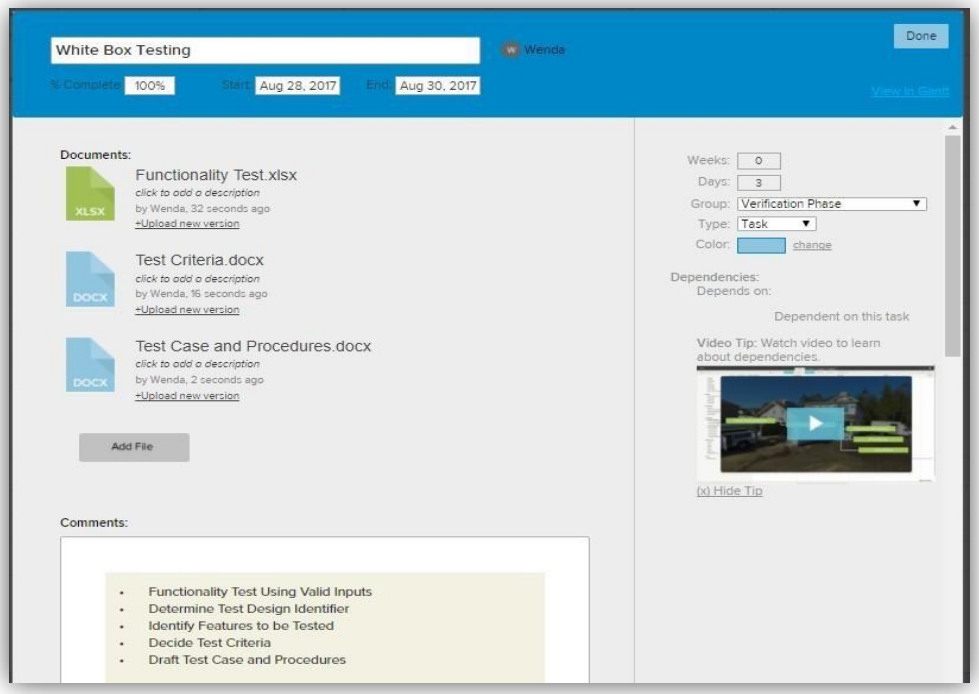


Figure 18. White Box Testing Task Info

1. **Code Coverage Analysis**
   Measure how much of the source code is executed during testing. This includes statement coverage, branch coverage, and path coverage to ensure all possible execution flows are tested.
2. **Control Flow Testing**
   Verify the logical flow of the program by testing loops, conditions, and decision-making structures within modules such as login, feedback submission, and concern categorization.
3. **Data Flow Testing**
   Examine how data moves through the system, ensuring variables are properly initialized, used, and updated. This prevents issues like uninitialized variables or incorrect data propagation.
4. **Unit Testing**
   Test individual modules (e.g., authentication, feedback form validation, reporting functions) to confirm that each performs correctly in isolation.
5. **Integration Testing**
   Validate the interaction between modules, ensuring that data flows correctly between the front-end, back-end, and database layers.
6. **Security Testing**

Check for vulnerabilities in the code, such as SQL injection risks, improper input validation, or weak session handling.

7. **Performance Testing**
   Assess the efficiency of algorithms and database queries to ensure the system can handle multiple concurrent users without degradation.
8. **Documentation**
   Record test cases, coverage reports, and results for future reference and continuous improvement.

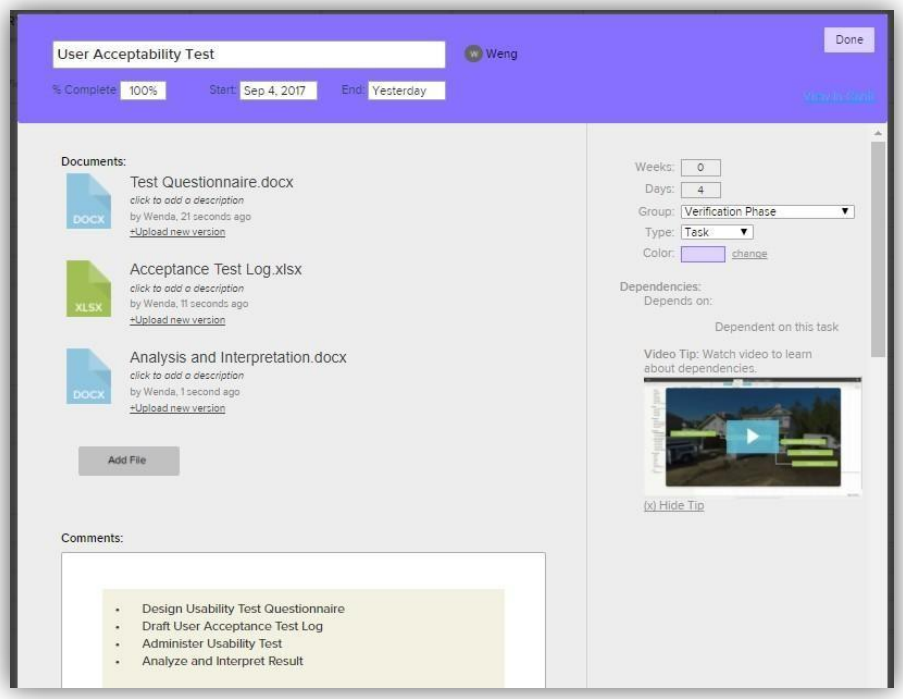


Figure 19.User Acceptability Test Task Info

1. **Test Planning**
   Define the scope, objectives, and criteria for acceptability. Identify user groups (students, teachers, administrators) and prepare test scenarios that reflect actual school processes.
2. **Test Environment Setup**
   Configure a controlled environment that mirrors the production system, including access credentials, sample data, and reporting modules.
3. **Scenario Development**

Create realistic scenarios such as submitting feedback, categorizing concerns, tracking resolutions, and generating reports. These scenarios simulate everyday use cases.

4. **Test Execution**
   Selected users perform tasks within the system. Their interactions are observed to assess usability, accuracy, and efficiency.
5. **Feedback Collection**
   Gather qualitative and quantitative feedback from participants regarding ease of use, clarity of workflows, and satisfaction with system performance.
6. **Issue Identification**
   Document any problems encountered, such as confusing navigation, slow response times, or missing features.
7. **Resolution and Retesting**
   Address identified issues through corrective actions. Retest the system to confirm that fixes meet user expectations.
8. **Approval and Sign-Off**
   Obtain formal approval from stakeholders, confirming that the system is acceptable for deployment in the school environment.

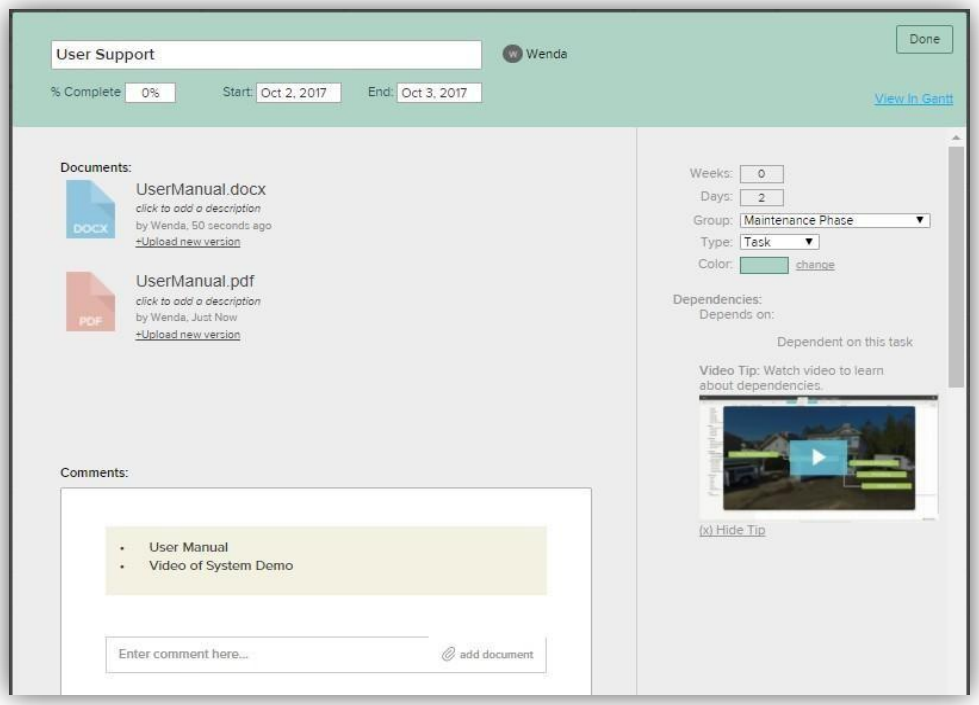


Figure 20. User Support Task Info

1. **Help Desk Support**
   Establish a dedicated support team or help desk to respond to user queries, troubleshoot issues, and provide immediate assistance.
2. **User Training**
   Conduct training sessions, workshops, or tutorials for students, teachers, and administrators. Focus on system navigation, feedback submission, concern tracking, and report generation.
3. **Documentation and Guides**
   Provide user manuals, FAQs, and quick reference guides to help users understand system features and resolve common issues independently.
4. **Feedback Collection**
   Gather user feedback on system usability and support services. Use this input to improve both the system and the support process.
5. **Issue Tracking**
   Implement a ticketing system to log, monitor, and resolve user-reported problems systematically.
6. **System Updates Communication**
   Notify users about new features, bug fixes, or system changes through announcements, emails, or in-app notifications.

7. **Continuous Improvement**
   Regularly review support performance, update training materials, and refine support strategies to enhance user satisfaction.

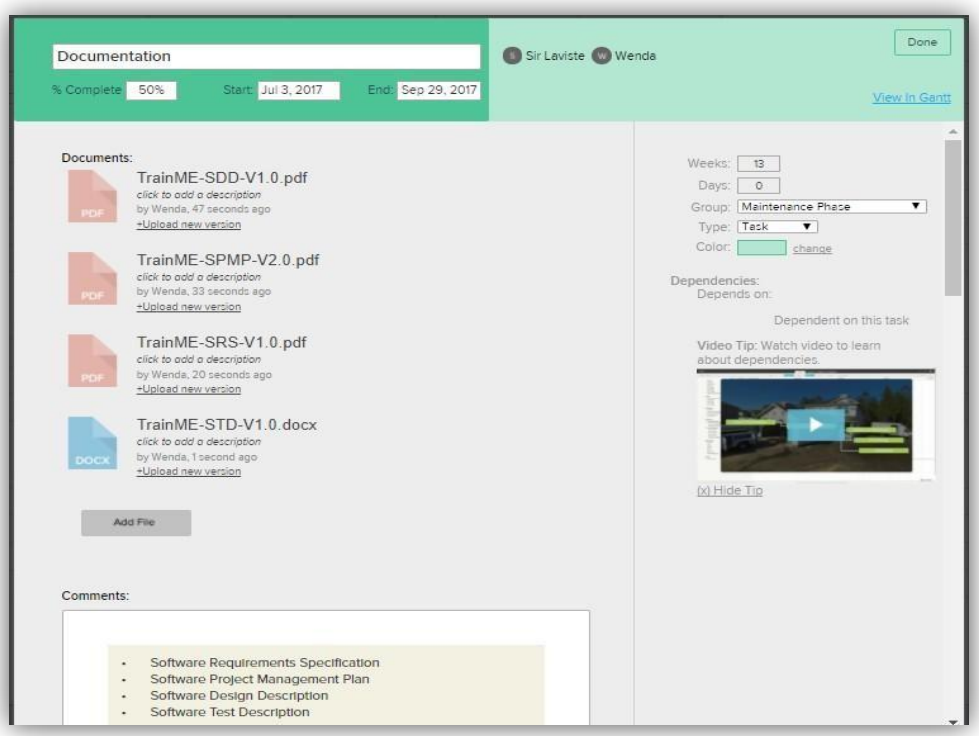


Figure 21. Documentation Task Info

1. **System Configuration Documentation**
   Detail server setup, software dependencies, and environment configurations to ensure smooth deployment and troubleshooting.
2. **Testing Documentation**
   Include test plans, test cases, coverage reports, and results from verification and user acceptance testing. This validates system reliability and performance.

3. **Maintenance Documentation**
   Provide logs of corrective, adaptive, perfective, and preventive maintenance activities. This ensures transparency and supports continuous improvement.
4. **Training Materials**
   Develop tutorials, presentations, and workshops to train users and administrators on system usage and best practices.
5. **Version Control and Updates**
   Track system versions, updates, and changes. Document release notes to inform users of new features, bug fixes, or enhancements.

5.2.3. Resource allocation

Table 7 shows the relationship between the tasks and resources needed along with the tools required. Table 8 reflects the main and supplemental resources needed for each project phase.

Table 7. Tasks-Resources-Tools Allocation Plan

Tasks/Phase	**Resources Needed**			
	Analyst	Developer	Document Controller	QA/Adviser
Requirements Analysis	Project Charter, Interview Guide, Questionnaires, Document samples		Office productivity tools, project management tools	
Design		Programming language packages, I/O devices, networking devices	Office productivity tools, project management tools	
Implementation		Programming language packages, I/O devices, networking devices	Office productivity tools, project management tools	
Verification and Validation (testing)	Office productivity tools, project management tools		Office productivity tools, project management tools	Test Instruments, Quality Assurance Test Logger, User Acceptance Test Logger

Maintenance	Administrative permission and support	Administrative permission and support	Office productivity tools, project management tools	

Table 8. Resource Allocation Plan

Task/Phase	Main Resource	Supplemental Resources
Requirements Analysis	Wenda D. Panes	Engr. Ralph Laviste
Design	Wenda D. Panes	Engr. Ralph Laviste
Implementation	Wenda D. Panes	Engr. Ralph Laviste
Verification and Validation (testing)	Wenda D. Panes	Engr. Ralph Laviste
Maintenance	Wenda D. Panes	Engr. Ralph Laviste

5.2.4. Budget allocation

The system's hardware budget allocation is shown in Table 9. It reflects the hardware specification, purpose and cost. The personnel budget allocation is reflected in the succeeding table which reflects the number of days and cost per day for each task performed by the developer who assumes different roles.

Table 9. Hardware Budget Allocation

Name	Specification	Purpose	Cost

Laptop Computer	OS: Microsoft Windows 10 Home System Type: x64-based PC Processor: Intell Corel i7-7500U CPU @ 2.70GHz, 2901 Mhz, 2 Core(s), 4 Logical Processor(s) Installed RAM: 8.00 GB SDD: 1TB	• Project software development • Project documentation • Server simulator	*₱46,000.00*
Broadband Internet	Bandwidth: up to 10Mbps Data Allocation: up to 400GB per month	• Web hosting services • Research and resource generation	*₱6,000.00*
Consumables		Supplies	*₱2,000.00*
		Total	***₱54,000.00***

Table 10. Personnel Budget Allocation

Project Task/Phase	Analyst		Developer		Technical Writer/QA		**Total Cost**
	No. of Days	**Cost (P300/day)**	**No. of Days**	**Cost (P500/day)**	**No. of Days**	**Cost (P300/day)**	
Project Initiation							P1000.00
Project Planning							P5000.00
Requirements Analysis	7	P2100.00			5	P1500.00	P3700.00
Design	10	P3000.00	8	P4000.00	5	P1500.00	P8500.00
Implementation	3	P900.00	30	P15000.00	2	P600.00	P16500.00
Verification and Validation (testing)	5	P1500.00			10	P3000.00	P4500.00
Maintenance	1	P300.00	1	P500.00	20	P6000.00	P7300.00

Project Monitoring & Controlling							P2000.00
Project Closing							P2000.00
						Total	**P50000.00**

5.3. Control plan

This clause discusses the control mechanisms employed in project development. It includes the requirements control plan, schedule control plan, budget control plan, quality control plan, reporting plan, metrics collection plan, risk management plan, and project closeout plan.

5.3.1. Requirements control plan

The project manager will be responsible for collecting and analyzing the software defects, package cyclical dependencies, and reviewing the code for comments. A metrics report will then be prepared and a resolution to any problem found will be initiated with the blessing of the adviser.

The project manager will also be responsible for determining if the product software truly meets the requirements. Findings will immediately be analyzed and fixes will be initiated to software that does not meet its stated requirements. This includes ensuring that the product software does not include requirements that are outside the scope of what the stakeholders has agreed to.

5.3.2. Schedule control plan

Actual start and finish dates will be recorded as progress occurs on the project. These dates will be compared to the original baseline to confirm that project deliverables and milestones will be completed in accordance with the project plan. The Project manager will do the monitoring on a weekly basis. The group along with stakeholders will be involved with decisions regarding the reassignment of resources or changes in priority.

5.3.3. Budget control plan

The allocated budget for each task will be strictly followed. Every purchase or expenses will be supported with receipts or related document. A record of debits and credits will be maintained in Microsoft Excel file. A control mechanisms will be used to measure the cost of work completed, compare planned cost to budgeted cost, and implement corrective action when actual cost does not conform to budgeted cost. The budget control plan shall

specify the intervals at which cost reporting will be done and the methods and tools that will be used to manage the budget. The budget plan should include frequent milestones that can be assessed for achievement using objective indicators to assess the scope and quality of work products completed at those milestones. A mechanism such as earned value tracking should be used to report the budget and schedule plan, schedule progress, and the cost of work completed.

5.3.4. Quality control plan

Actual start and finish dates will be recorded as progress occurs on the project. These dates will be compared to the original baseline to confirm that project deliverables and milestones will be completed in accordance with the project plan. The Project manager will do the monitoring on a weekly basis. The group along with stakeholders will be involved with decisions regarding the reassignment of resources or changes in priority.

5.3.5. Reporting plan

Meetings will be held twice a week on Wednesday and Saturday to determine progress work on the deliverables. The project manager will prepare a weekly status reports which will include percent completed, estimated completion date, and effort expended figures for the task currently at hand.

Major problems will be reported as soon as discovered and additional meetings will be held, if necessary, to handle special issues (such as milestone slippage). Any problems encountered and changes made will be reported immediately to the project adviser.

More importantly, email and phone communications will be mandatory. Project manager and adviser will constantly communicate through Gmail or Messenger group. All products of the project will be uploaded to the Google drive and will be shared with the group. The Messenger group will serve as a configuration control tool for the project. Email will be the preferred method of non-critical communication among team members.

5.3.6. Metrics collection plan

The project manager will maintain a Concurrent Versioning System (CVS) repository on her personal computer. When the coding phase starts, every Wednesday by 11:00 PM the developers will deliver software updates that compile with no errors to the Messenger group and Google drive. Along

with the software updates, the requirements the updates fulfilled will be carefully noted. Any changes will be committed to the CVS repository by Saturday morning 8:00 AM. All builds created for system testing and product release will be performed directly from the CVS computer. The resulting metrics and a list of requirements fulfilled by the build will be put into a release file. Each build and release file will be zipped up and uploaded to the Messenger group and Google drive for the adviser to review. Each distribution build will contain source code as well as all executable code, libraries, etc. necessary to deploy and run the application. The project lead will deploy builds for system test on her laptop and make these builds available for other team members to test either via the Internet or by allowing other team members to use her laptop for testing. In addition, she will deploy builds on the CVS computer and make that computer available for members to test.

.

5.3.7 Risk management plan

The main risk to project success is that each phase of the project will take more time than anticipated. This risk is believed to be high since the project "team" has never worked together before or produced a product that meets the same or similar requirements and therefore, has no historical data on which to base the estimates of effort. The estimates of effort are based in large part on educated guesses. If a phase is found to take significantly longer than expected, then the team will determine if the scope should be reduced or if more resources can be assigned to the particular task(s) in order to meet the deadline for the final product.

5.3.8 Project closeout plan

All document deliverables must be uploaded to the Messenger group and Google drive 48 hours before their deadline so that all stakeholders can review the document and make corrections and suggestions. All corrections and suggestions must be emailed to every member of the group 24 hours before the deadline so that the member coordinating the document can make corrections in time for the deadline. All team members must sign off the final document 12 hours before the deadline.

6. Technical process plans

This section of the TrainME SPMP specifies the development process model, the technical methods, tools, and techniques to be used to develop the various work products; plans for establishing and maintaining the project infrastructure; and the product acceptance plan.

6.1 Process Model

The main process model for this project will be a variation of the waterfall model as shown in Figure 22, which includes overlapping phases, feedback loops, and prototypes. Subsequent phases will begin before the previous phase is complete. Feedback from the subsequent phase will be applied to the previous phase before it is complete. The diagram below shows the major phases as well as the deliverables for this model. The deliverables shown with a dashed outline are internal deliverables, meaning they won't be delivered to the client. These deliverables include a static Interface Prototype and a Functional Prototype. In this diagram, the User Guide and Product Reference Manual deliverables are considered part of the Product Software deliverable.

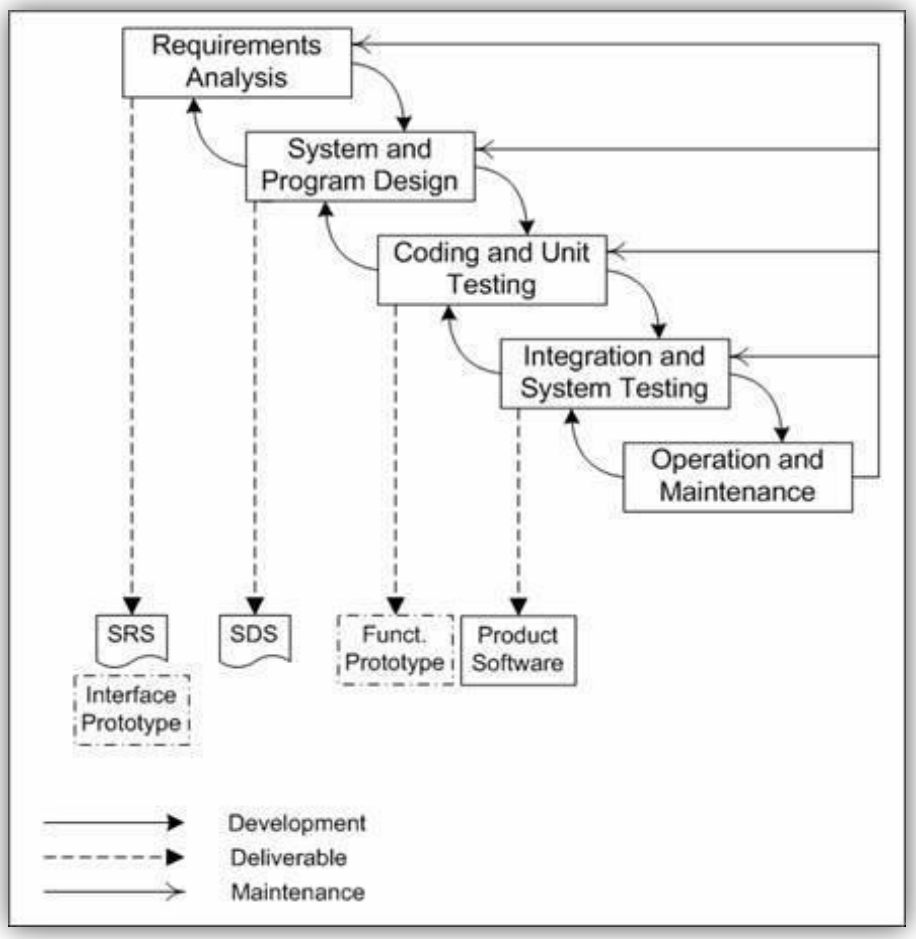


Figure 22. Software Development Waterfall Model

1. **Requirement Analysis**
   Collect and document user and system requirements from stakeholders (students, teachers, administrators). Define functional and non-functional needs such as feedback submission, concern categorization, reporting, usability, and security.
2. **System Design**
   Translate requirements into system architecture. Develop database schemas, front-end layouts, and back-end workflows. Ensure design supports scalability and integration with school IT infrastructure.
3. **Implementation**
   Develop the system in two layers: ○ **Front-End Implementation**: Build user interfaces for students, teachers, and administrators.
   ○ **Back-End Implementation**: Develop APIs, business logic, and database integration.
4. **Verification**

Conduct rigorous testing, including white box testing, user acceptability testing, and defect management. Ensure the system meets requirements and functions correctly.

5. **Deployment**
   Roll out the system to the school environment. Provide training, documentation, and initial user support to ensure smooth adoption.
6. **Maintenance**
   Perform corrective, adaptive, perfective, and preventive maintenance. Provide ongoing user support, issue tracking, and system updates to sustain reliability and usability.

Overview of Phases

Concern track uses management tool as a guide in project development. The five phases of the Waterfall model are laid out as well as the specific tasks for each phase. The detail discussion and description for each phase are narrated as follows:

Requirements Analysis Phase

During this phase, the project manager conducted user and system requirements analysis. Required documents such as Training Needs Assessment Survey Form, Training Designs, Training Programs, Memoranda of Agreement/Understanding were gathered. Then, business rules were determined by looking into the Extension Manual and Procedures in the Conduct of Extension Services from the OESCD. To further understand the requirements, a series of interview were conducted with the Research and Extension Chair and Staff, faculty- trainers as well as selected clients. For an in-depth understanding of processes, research on mandates, training needs, clients/beneficiaries, and funding agencies was conducted. Further, activities before, during and after the training were keenly observed.

To determine system needs, the project manager drafted the External Interface Requirements involving user, hardware, software and communication requirements. Functional requirements in the form of use case diagrams were designed using an appropriate UML designing software. Consequently, performance requirements were identified.

System and Program Design Phase

Prototyping, class diagram design, data flow diagram design as well as database design were performed in this phase. To produce a low-fidelity feel of the system to be developed, a paper prototype was created and use case diagrams were designed based on the paper prototype. This was

subjected to usability test to target users. After which, class diagrams were created following the model-view-controller framework. Subsequently, sequence diagrams were designed using the Astah Community software tool. The data inputs, processes and outputs were determined with the help of data flow diagrams. Finally, the normalized and relational system database was designed using MySQL.

Coding and Unit Testing

In this phase of the waterfall model, four major sub-tasks were performed namely: front-end implementation, database implementation, back-end implementation and network set-up. The project manager who also assumes the roles of GUI designer and project developer started with implementing the system admin interface, trainer interface and client interface using the HTML, CSS, java script and bootstrap technologies. Appropriate and required queries were coded to extract data from the database. As to how data should behave given different transactions, such as create, read, update, send, store, print, organize, compute, match, etc. were coded meticulously, and logically following the syntactic rules of PHP programming language. Since there are three different types of users with unique system accessibility permissions, a network was set-up to have the system run simultaneously on at least three devices. Specific codes were embedded to enable multi-user web access of the system.

Integration and System Testing Phase

This phase is also known as VV and the project manager who is also a QA tester performs three major tasks. First, White Box testing where the system functionality is tested and this is done by feeding the system with valid inputs. But before that, test case and procedures are first drafted, test criteria are decided upon, system features to be tested are identified and test design identifier is determined. Second, Black Box testing by which invalid inputs are used to test the system functionality. A test log is drafted for this task. Finally, the system is subjected to User Acceptability testing. This is done by first designing test questionnaires, then drafting user acceptance test log, determining respondents, administering the usability test and finally analyzing and interpreting the results.

Operation and Maintenance

This phase requires four major tasks each of which contain a number of specific activities. During the Training and Dissemination, system users must be trained on how to manipulate and navigate the system. Separate trainings will be conducted for admins, trainers, and clients.

Permission from proper authorities will be sought and training module will be prepared.

A considerable amount of time is allotted for the documentation task which occurs all throughout the project development but will formally be organized on a specified time as reflected in the WBS. Document deliverables include Software Requirements Specification which lays out functional and non-functional requirements, and may include a set of use cases that describe user interactions that the software must provide; Software Project Management Plan that comprises of a number of activities, which contains planning of project, deciding scope of software product, estimation of cost, etc.; Software Design Description that servers as the overall guidance to the architecture of the software project; and Software Test Description that describes the test preparations, testcases, and test procedures to be used to perform qualification testing of a Computer
Software Configuration Item (CSCI) or a software system or subsystem

User support tasks are also performed in this phase. A User Manual is drafted to serve as reference for users on the vital information about the system as well as how the system is operated. Installation requirements and dependencies are also reflected in this document. For more visual users, a video of the system walkthrough or demonstration will be developed. A website dedicated for inquiries request and response may also be designed.

Finally, Nice-to-Haves features may be incorporated if time permits.

6.2 Methods, tools, and techniques

Concern track will be developed using a variation of the waterfall model, which includes overlapping phases, feedback loops, and prototypes. For example, before the requirements analysis phase is completed the program design phase will begin. Before the program design phase is completed, the program coding phase will begin and an early prototype will be developed. Feedback from later phases will be incorporated into earlier phases before those phases are complete. Lessons learned from the early prototype will be incorporated into the final product.

Concern track will be developed on Personal Computers (PC) compatible with the x86 architecture running either Microsoft Windows 8 or Microsoft Windows 10. The developer used Sublime text for coding, integration, compiling, and debugging. Apache Ant will be used for building and deployment during unit, integration, and system testing.

The database for will run on MySQL (http://www.mysql.com). MySQL is the world's most popular open-source database. MySQL is freely available, easy to use, and cross-platform. If needed, the creators of MySQL also provide professional technical support at a Gantt (https://app.teamgantt.com) will be used to monitor the percentage of requirements that have been completed. Gantt is a free online project management tool ideal for project scheduling. Table 11 and Table 12 summarizes the methods and techniques and tools respectively.

Table 11. Methods and Techniques Plan

Category	Methods and Techniques
Requirements	Document Sampling Interviews Brainstorming Research Observation
Formal Specification and Analysis	Formal models using UML to model structural aspects of the requirements and design Use cases to define requirements. Class Diagram to define objects.
Prototype	GUI Transition prototypes to validate a technical or design. This is different from the development cycle's construction effort, which is geared towards the final deliverable.

Table 12. Software Tools Plan

Category	Tools
Operating System	Windows 10
Server	XAMPP
Databases	Apache, MySQL

UI Design	Bootstrap, HTML, CSS, Java script
Programming Language	PHP
Editor	Sublime
Browser	Google Chrome, Mozilla Firefox, Internet Explorer
Document	Microsoft Word
Project Planning and tracking	Team Gantt, Microsoft Excel, Mind View
UML Diagrams	Astah Community

6.3 Infrastructure Plan

Concern track project is developed in a network environment. A provision for at least 3 workstations with I/O peripherals and networking devices is needed. Resources in the computer laboratory or faculty room may be utilized for two hours upon request preferably during weekdays after class. Since the project manager will be working inside the campus premises, provision for security and access to electrical facilities may be extended to her.

6.4 Product Acceptance Plan

The main stakeholder of the system is the OESCD of ISCOF-SEC. A Memorandum of Agreement must be signed between two parties to express agreement on the terms and conditions on the utilization of the system. The agreement must adhere to Intellectual Property policies and guidelines.

7. Supporting process plans

This section contain plans for the supporting processes that span the duration of the software project. These plans shall include configuration management, verification and validation, and software documentation plans. Other plans required in the IEEE format for SPMP are not applicable to this particular project.

7.1. Configuration management plan

Actual start and finish dates will be recorded as progress occurs on the project. These dates will be compared to the original baseline to confirm that project deliverables and milestones will be completed in accordance with the project plan. The Project manager will do the monitoring on a weekly basis. The group along with stakeholders will be involved with decisions regarding the reassignment of resources or changes in priority.

7.2. Verification and validation plan

This sub clause of SPMP contains the verification and validation plan for the Concern track software project to include scope, tools, techniques, and responsibilities for the verification and validation work activities as reflected in Table 13. The organizational relationships and degrees of independence between development activities and verification and validation activities are also specified.

Several tasks collectively make up continuing activities that go across the different life cycle phases. The general activities are traceability analysis, evaluation, interface analysis, and testing. These activities are horizontal threads that ties together the subsequent phase activities and allow verification to be more effectively conducted.

Traceability analysis

The traceability is the ability to identify the relationships between originating requirements and their resulting system features. It permits tracking forward or backward through the network of interrelationships that are created as requirements are decomposed and refined through a system's life cycle. Traceability allows verification of the properties set forth in the concept and that requirement specifications have been carried forward to the design specification, implemented in the code, included in the test plan and cases, and provided to the customer and user in the resulting system.

Evaluation

Evaluation ascertains the value or worth of an item and help to assure that a system meets its specifications. Evaluations are preformed by many persons across all life cycle phases, on both interim and final software products, and may be either a comprehensive or selective assessment of a system. Evaluations are used through all phases and for all type of software products, including user documents, manuals, and other project documents. These may be of many forms, such as text or graphic representations, and in various media.

Interface analysis

When information is passed across a boundary, there is always the possibility of losing some information or alerting the information content. The task of interface analysis serves to ensure the completeness, accuracy, and consistency of these interfaces. Interface requirements at the design and implementation phases should be identified analyzed at the functional, physical, and data interface level. The goal of interface analysis is to evaluate the specific software deliverables (e.g., requirements, design, code) for correct, consistent, complete, and accurate interpretation of the interface requirements.

Testing

In the context of software verification and validation, testing can be defined as the testing that is performed in support of the V&V objectives. These objectives may differ from those of the developer. Testing is performed at several points in the life cycle, starting from the requirement phase up to the test phase. The various test activities are listed below:

Component Testing
Testing conducted to verify the implementation of the design for one software elements or a collection of software elements

Integrating Testing
An orderly progression of testing in which software elements, hardware elements, or both are combined and tested until the entire system has been integrated.

System Testing
The process of testing an integrated hardware and software system to verify that the system meets its specified requirements

Acceptance Test
Formal testing conducted to determine whether or not a system satisfies its acceptance criteria and to enable the customer to determine whether or not to accept the system

Table 13. Verification and Validation Plan

Phase	V&V Input	V&V Tasks	V&V Output
Requirements	SRS Interface requirements documentation User documentation	Requirements traceability analysis Requirements evaluation Requirements interface analysis Test plan generation	Requirements phase tasks reporting Test plan System Acceptance
Design	SRS Interface requirements documentation Interface design documentation User documentation	Design traceability analysis Design evaluation Interface analysis Test plan generation Test design generation	Design phase task reporting Test plan Component Integration Test design Component Integration System acceptance
Implementation	Source Code listing Executable code Interface design documentation User documentation	Code traceability analysis Code evaluation Interface analysis Documentation evaluation Test case generation Test procedure generation Component test execution	Implementation phase task reporting Test cases Component Integration System Acceptance Test procedure

			Component Integration - System
Test	Source code listing Executable code User documentation	Test procedure generation Integration test execution System test execution Acceptance test execution	Test phase task reporting Test procedure Acceptance Anomaly report V&V phase summary
Installation and Checkout	Installation package	Installation configuration audit V&V final report generation	Installation and checkout phase task reporting

7.3. Documentation plan

There are a number of documents that will be produced during the lifetime of the Concern track project. All documents are responsibilities of the developer. The list of documents that will be created and maintained include:

- Software Project Management Plan (SPMP)
- Software Requirements Specification (SRS)
- Software Design Documentation
- Software Test Documentation
- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Entity Relationship Diagram (ERD)
- Test log and test results